

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
CARRERA DE CIENCIAS DE LA COMPUTACIÓN



**IMPLEMENTACIÓN DE UN PROCESADOR PIPELINE EN BASE A UN SINGLE
CYCLE**

AUTORES

Marco Wanly Obregón Casique
Ferrel Valentino Infante Garcia
Yuri Abel Escobar

ASESOR(ES)

Williams Valencia, Carlos Eduardo

Lima – Perú

2026

ÍNDICE

1. Introducción

2. Marco Teórico

3. Implementación de las instrucciones comprimidas a 16 bits

3.1. Filosofía de diseño y módulo del Descompresor

3.1.1 Naturaleza combinacional del descompresor

3.1.2 Detección de compresión

3.1.3 Mapeo de registros restringidos (3 bits a 5 bits)

3.2 Clasificación de las 10 Instrucciones Implementadas

3.3. Detalle de la implementacion de las instrucciones segun su tipo

3.3.1. Operaciones Aritméticas con Inmediato (c.addi y c.lui)

3.3.2. Operaciones Aritméticas Registro-Registro (c.add y c.sub)

3.3.3. Operaciones Lógicas de Bits (c.and, c.or, c.xor)}

3.3.4. Instrucciones de Desplazamiento (c.slli, c.srli, c.srai)

3.4. Cambios en el datapath

3.4.1 imem.v

3.4.2 if_stage.v

3.4.3 pipeline.v

3.4.4 regfile.v

3.4.5 Hazard unit no requirió cambios

4. Resultados

4.1 Programa de prueba

4.2 Análisis del waveform

5. Conclusiones

1. Introducción

La extensión C de RISC-V crea instrucciones de 16 bits que reemplazan a las de 32 bits más comunes, reduciendo el tamaño del código binario alrededor de 30% sin pérdida alguna. Cada una de las instrucciones tiene una contraparte de 32 bits que hacen exactamente lo mismo, por lo que basta con agregar un módulo descompresor que los traduzca antes de que avancen por el pipeline.

Las instrucciones de 16 y 32 bits se distinguen por sus dos bits menos significativos, esto tiene una consecuencia directa, al ejecutar una instrucción comprimida el PC va a avanzar 2 bytes en vez de 4.

Este informe corresponde a la entrega E2 del proyecto, e implementa diez funciones comprimidas sobre el procesador pipelined de la parte 1, explicamos cómo funciona el descompresor, la expansión de cada instrucción, los cambios en el datapath y la validación con un programa que mezcla instrucciones de 16 y 32 bits

2. Marco Teórico

La extensión 'C' no introduce un modo de operación separado, sino que amplía el ISA base permitiendo que las instrucciones de 16 y 32 bits se mezclen en un mismo flujo de ejecución. El ahorro de espacio viene de explotar patrones dentro del código en lugar de codificar cada instrucción de forma completa. Usamos immediatos pequeños esto restringe muchos de sus formatos a ocho registros de uso común (x8–x15, codificables en solo 3 bits en vez de 5) y reutiliza el registro destino como primer operando fuente ($rd = rs1$), evitando

codificar campos redundantes. Gracias a estas restricciones, cada instrucción comprimida admite una correspondencia uno a uno con una instrucción del ISA base, lo que permite soportar toda la extensión con un único bloque combinacional que traduce la instrucción de 16 bits a su equivalente de 32 bits antes de que esta avance por el pipeline, sin alterar las etapas posteriores ni la lógica de detección de hazards.

3. Implementación de las instrucciones comprimidas a 16 bits

3.1. Filosofía de diseño y módulo del Descompresor

3.1.1 Naturaleza combinacional del descompresor

El módulo decompressor.v es el núcleo de la extensión RVC. Se implementa como lógica combinacional pura, es decir no tiene entradas de reloj, no mantiene estado interno y su salida depende exclusivamente de la entrada actual de 16 bits. Para nuestra implementación, optamos por incorporarlo en la etapa Fetch (IF) del pipeline, justo después de la memoria de instrucciones (imem.v) y antes de que la instrucción ingrese al registro pipeline IF/ID.

El flujo de datos se da en el siguiente orden:

1	2	3	4
imem (lectura)	decompressor (expansión)	InstrF (32 bits)	IF/ID register

Tiene la función principal de leer la instrucción de 16 bits (halfword) desde la memoria de instrucciones y produce de manera instantánea su equivalente de 32 bits para que el resto del pipeline no perciba la diferencia entre formatos.

3.1.2 Detección de compresión

El criterio de detección está dado de acuerdo al estandar para instrucciones comprimidas, es decir, se logra evaluando los dos bits menos significativos (`instr16[1:0]`) que definen el quadrant de la instrucción. Si son iguales a `2'b11`, la instrucción es de 32 bits (formato RV32I estándar). Cualquier otro valor (`2'b00`, `2'b01`, `2'b10`) indica una instrucción comprimida de 16 bits.

Esta señal `is_compressed` se propaga a `if_stage.v` para dos propósitos:

```
assign is_compressed = (instr16[1:0] != 2'b11);
```

Seleccionar la salida: si está activa, se usa la instrucción expandida `DecomplInstrF`; si no, se usa la instrucción original `RawInstrF`.

Ajustar el PC: cuando es 1, el PC avanza +2 bytes; cuando es 0, avanza +4.

```
assign InstrF = IsCompressedF ? DecomplInstrF : RawInstrF;  
assign PCIncF = IsCompressedF ? (PCF + 32'd2) : (PCF + 32'd4);
```

3.1.3 Mapeo de registros restringidos (3 bits a 5 bits)

Los formatos RVC utilizan campos de 3 bits para referirse a registros en la mayoría de las instrucciones (formatos CA, CB, CL, CS). Esto permite empaquetar la instrucción en 16 bits, pero limita el rango direccionable a 8 registros. La especificación RISC-V mapea estos 3 bits al rango `x8–x15`:

```
wire [4:0] rd_f = instr16[11:7]; // rd completo (CR, CI, CSS)  
wire [4:0] rs2_f = instr16[6:2]; // rs2 completo (CR, CSS)
```

La concatenación `{2'b01, reg_3bits}` produce un registro de 5 bits completo en el rango 8 a 15 (`0b01000` a `0b01111`).

Excepcionalmente, algunas instrucciones RVC como `c.add` y `c.slli` utilizan el formato CR/CI, que emplea campos de 5 bits directos y puede direccionar cualquier registro del banco (`x0–x31`):

```
wire [4:0] rd_f = instr16[11:7]; // rd completo (CR, CI, CSS)  
wire [4:0] rs2_f = instr16[6:2]; // rs2 completo (CR, CSS)
```

3.2 Clasificación de las 10 Instrucciones Implementadas

La siguiente tabla resume las 10 instrucciones RVC implementadas, clasificadas por su quadrante, formato RVC y equivalente en RV32I:

Instrucción RVC	Quadrant	Formato RVC	Equivalente 32 bits	Tipo
c.addi rd, imm	01	CI	addi rd, rd, imm	Aritmética con inmediato
c.lui rd, nzimm	01	CI	lui rd, nzimm	Carga de constante
c.add rd, rs2	10	CR	add rd, rd, rs2	Aritmética registro-registro
c.sub rd', rs2'	01	CA	sub rd', rd', rs2'	Aritmética registro-registro
c.and rd', rs2'	01	CA	and rd', rd', rs2'	Lógica
c.or rd', rs2'	01	CA	or rd', rd', rs2'	Lógica
c.xor rd', rs2'	01	CA	xor rd', rd', rs2'	Lógica
c.slli rd, shamt	10	CI	slli rd, rd, shamt	Desplazamiento
c.srli rd', shamt	01	CB	srli rd', rd', shamt	Desplazamiento
c.srai rd', shamt	01	CB	srai rd', rd', shamt	Desplazamiento

En la tabla mostrada, debemos aclarar que debe ser leída considerando lo siguiente:

1. CI: Inmediato compacto, un registro fuente-destino.
2. CR: Dos registros completos (5 bits cada uno).
3. CA: Dos registros restringidos (3 bits cada uno, x8–x15) más funct.

4. CB: Un registro restringido más un inmediato desplazamiento.

3.3. Detalle de la implementación de las instrucciones segun su tipo

3.3.1. Operaciones Aritméticas con Inmediato (c.addi y c.lui)

Para las presentes operaciones aritméticas, su equivalente en 32 bits se define como sigue:

-

Instrucción RVC	Equivalente en 32 bits
c.addi rd, imm	addi rd, rd, imm
c.lui rd, nzimm	lui rd, nzimm

Para c.addi detallamos su configuración:

Quadrant	01 Funct3: 000 Formato: CI
Campos	rd = instr[11:7]

El inmediato de 6 bits se extrae de los campos instr[12] (bit de signo) e instr[6:2] (magnitud). Se sign- extiende a 12 bits para formar el campo inmediato I-type de la instrucción de 32 bits. El registro rd actúa simultáneamente como fuente y destino, reflejando la semántica de la instrucción comprimida que no codifica un registro fuente por separado.

A continuación procedemos a mostrar la expansión bit a bit para c.addi x8, 3 (0x040D):

instr16[15:0]	0000_0100_0000_1101
instr16[12]	0 (inmediato positivo)
instr16[6:2]	00011 (3 en decimal)
rd (instr[11:7])	01000 (x8)
instr32	000000000011_01000_000_01000_0010011 = 0x00340413

	= addi x8, x8, 3
--	------------------

En formato de instrucción de 32 bits, se especifica de la siguiente manera.

```
instr32 = {{6{instr16[12]}}, instr16[12], instr16[6:2],
          rd_f, 3'b000, rd_f, 7'b0010011};
```

Donde el opcode 0010011 (I-type), funct3 000 (add) y el inmediato sign-extendido son heredados de la instrucción addi de RV32I.

Asimismo respecto a la instrucción c.lui procedemos a detallarla análogamente

Quadrant	01 Funct3: 011 Formato: CI
Campos	rd = instr[11:7]

A diferencia de c.addi, el inmediato de 6 bits {instr[12], instr[6:2]} se mapea a los bits [17:12] del campo U-type de 20 bits. Los bits superiores ([31:18]) se obtienen por sign-extension. Esto permite cargar constantes de hasta 17 bits (con signo) en la parte alta de un registro.

Expansión bit a bit para c.lui x15, 2 (0x6789):

instr16[15:0]	0110_0111_1000_1001
instr16[12]	0 (signo positivo)
instr16[6:2]	00010 (2 en decimal)
rd (instr[11:7])	01111 (x15)
nzimm[17:12]	{0, 00010} = 000010
instr32[31:12]	{14{0}}, 0, 00010 = 0x0000_2000
instr32	0x0000_2000_01111_0110111 = 0x0000_2F37
	= lui x15, 0x2000 (x15 = 8192)


```
instr32 = {{14{instr16[12]}}, instr16[12], instr16[6:2],
          rd_f, 7'b0110111};
```

El opcode 0110111 (LUI) y el formato U-type hacen que el valor cargado sea $nzimm \ll 12$, equivalente a multiplicar el inmediato por 4096.

3.3.2. Operaciones Aritméticas Registro-Registro (c.add y c.sub)

c.add rd, rs2 equivale a add rd, rd, rs2

- Quadrant: 10 | Funct3: 100 (instr[15:13]) | Funct4: 1001 (instr[15:12]) | Formato: CR
- Campos: rd = instr[11:7], rs2 = instr[6:2] (ambos de 5 bits)

Es una de las pocas instrucciones RVC que permite usar cualquier registro del banco como fuente y destino, gracias a su formato CR que asigna 5 bits completos para cada campo. Se distingue de c.jr y c.jalr porque comparten el mismo cuadrante y funct3, pero c.add tiene instr[12]=1 y rs2≠0.

Expansión:

```
instr32 = {7'b0000000, rs2_f, rd_f, 3'b000, rd_f, 7'b0110011};
```

La concatenación reconstruye una instrucción R-type completa:

- funct7 = 0000000 (ADD, no SUB)
- rs2 = rs2_f (5 bits)
- rs1 = rd_f (el mismo registro es fuente y destino)
- funct3 = 000
- rd = rd_f
- opcode = 0110011 (R-type)

c.sub rd', rs2' equivale a sub rd', rd', rs2

● Quadrant	● 01 Funct3: 100 Formato: CA
● Campo discriminador	● {instr[12], instr[6:5]} = 3'b000
● Registros	● rd' = rs1' = {2'b01, instr[9:7]}, rs2' = {2'b01, instr[4:2]} (x8–x15)

A diferencia de c.add, usa el formato CA con registros restringidos a x8–x15. La clave de la expansión es forzar funct7 = 0100000, que es el bit que la ALU de RV32I utiliza para diferenciar SUB de ADD en las instrucciones R-type.

```
instr32 = {7'b0100000, rs2_p, rs1_p, 3'b000, rs1_p, 7'b0110011};
```

El funct7 en 0100000 (con el bit 30 en 1) indica a la ALU que realice una resta en lugar de una suma. El funct3 = 000 y el opcode = 0110011 completan la instrucción R-type equivalente a sub.

3.3.3. Operaciones Lógicas de Bits (c.and, c.or, c.xor)}

- **Quadrant:** 01 | **Funct3:** 100 | **Formato:** CA
- **Campo discriminador:** {instr[12], instr[6:5]}:
 - 001 mapea a xor (funct3 = 100)
 - 010 mapea a or (funct3 = 110)
 - 011 mapea a and (funct3 = 111)
- **Registros:** rs1' = {2'b01, instr[9:7]}, rs2' = {2'b01, instr[4:2]} (x8–x15)

Las tres instrucciones comparten el mismo formato CA y se diferencian exclusivamente por el valor del campo funct3 en la instrucción expandida. El decompressor traduce el discriminador {instr[12], instr[6:5]} al funct3 correspondiente de RV32I.

```
// c.sub: funct3=000, funct7=0100000 (ya visto)
```

```
3'b000: instr32 = {7'b0100000, rs2_p, rs1_p, 3'b000, rs1_p, 7'b0110011};
```

```
// c.xor: funct3=100
```

```

3'b001: instr32 = {7'b0000000, rs2_p, rs1_p, 3'b100, rs1_p, 7'b0110011};

// c.or: funct3=110
3'b010: instr32 = {7'b0000000, rs2_p, rs1_p, 3'b110, rs1_p, 7'b0110011};

// c.and: funct3=111
3'b011: instr32 = {7'b0000000, rs2_p, rs1_p, 3'b111, rs1_p, 7'b0110011};

```

El funct7 = 0000000 para las tres operaciones lógicas (solo SUB requiere funct7 = 0100000). El opcode = 0110011 (R-type) se mantiene constante. La ALU del pipeline recibe el funct3 directamente desde el campo decodificado de la instrucción expandida, por lo que ejecuta la operación correcta sin ninguna modificación en la etapa EX.

3.3.4. Instrucciones de Desplazamiento (c.slli, c.srli, c.srai)

c.slli rd, shamt equivale a *slli rd, rd, shamt*

- **Quadrant:** 10 | **Funct3:** 000 | **Formato:** CI
- **Campos:** rd = instr[11:7] (5 bits, cualquier registro), shamt = instr[6:2]

A diferencia de las instrucciones de desplazamiento del cuadrante 01, c.slli usa registros de 5 bits completos (formato CI en cuadrante 10), permitiendo desplazar cualquier registro del banco.

```

instr32 = {7'b0000000, instr16[6:2], rd_f, 3'b001, rd_f,
7'b0010011};

```

La expansión reconstruye slli (I-type con funct3 = 001): el shamt se coloca en los bits [24:20] del campo funct7 (el bit 25 es el LSB de funct7 = 0), el registro fuente y destino es el mismo (rd_f), y el opcode 0010011 indica una operación ALU con inmediato.

c.srli rd', shamt y *c.srai rd', shamt*

- **Quadrant:** 01 | **Funct3:** 100 | **Funct2 (instr[11:10]):** 00 (srli), 01 (srai)

- **Formato:** CB
- **Registros:** $rs1' = \{2'b01, instr[9:7]\}$ (x8–x15), $shamt = instr[6:2]$

Ambas instrucciones difieren únicamente en el valor de `funct7`. El decompressor inspecciona `instr[11:10]` para decidir:

- **c.srli** (`instr[11:10] = 00`): desplazamiento **lógico** hacia la derecha. Usa `funct7 = 0000000`, que indica a la ALU que rellene con ceros.
- **c.srai** (`instr[11:10] = 01`): desplazamiento **aritmético** hacia la derecha. Usa `funct7 = 0100000`, que indica a la ALU que extienda el bit de signo.

```
// c.srli: funct7=0000000 (shift lógico)
instr32 = {7'b0000000, instr16[6:2], rs1_p, 3'b101, rs1_p, 7'b0010011};

// c.srai: funct7=0100000 (shift aritmético)
instr32 = {7'b0100000, instr16[6:2], rs1_p, 3'b101, rs1_p, 7'b0010011};
```

Ambas usan `funct3 = 101` (específico de desplazamientos a la derecha en RV32I) y `opcode = 0010011` (I-type ALU). La diferencia en `funct7` es la misma que usa el repertorio RV32I para distinguir `srli` de `srai`.

3.4. Cambios en el datapath

La incorporación de la extensión RVC no se limitó a la creación del módulo decompressor. Fue necesario modificar varios componentes del datapath del pipeline base para que pudieran manejar direcciones de memoria alineadas a 2 bytes, instrucciones de 16 bits en la memoria y un avance de PC variable. En total, cuatro módulos existentes recibieron modificaciones: la memoria de instrucciones (`imem.v`), la etapa de fetch (`if_stage.v`), el orquestador del pipeline (`pipeline.v`) y el banco de registros (`regfile.v`). El módulo de detección de hazards (`hazard_unit.v`), en cambio, no requirió alteración alguna, y en la última subsección se explicará por qué.

Cada una de las siguientes subsecciones presenta el problema que motivó el cambio, muestra el código antes y después de la modificación, y analiza el impacto funcional de la adaptación.

3.4.1 imem.v

En el pipeline base de la Parte 1, la memoria de instrucciones almacenaba palabras completas de 32 bits y el direccionamiento se realizaba mediante el índice `a[31:2]`, que ignora los dos bits menos significativos de la dirección. Esto solo permite lecturas alineadas a 4 bytes, que es suficiente para instrucciones RV32I pero no para RVC. Con la extensión comprimida, el PC puede apuntar a direcciones pares como `0x02`, `0x06` o `0x0A`, que no son múltiplos de 4. Para soportar estas direcciones, la memoria debe ser capaz de leer desde cualquier alineación a 2 bytes.

Código antes del cambio (pipeline base):

```
reg [31:0] RAM [0:63];  
assign rd = RAM[a[31:2]];
```

La memoria se declaraba como un arreglo de 64 palabras de 32 bits, direccionado únicamente con los bits `a[31:2]`. El bit `a[1]` era ignorado, impidiendo distinguir entre una dirección como `0x00` y otra como `0x02`.

Código después del cambio (con soporte RVC):

```
reg [15:0] RAM [0:127];  
wire [6:0] idx = a[7:1];  
assign rd = {RAM[idx + 7'd1], RAM[idx]};
```

El cambio consiste en tres modificaciones. Primero, el ancho de cada entrada se reduce de 32 bits a 16 bits (halfwords), y la profundidad se duplica de 64 a 127 entradas, manteniendo el mismo espacio total de direccionamiento. Segundo, el índice de lectura ahora usa `a[7:1]` en lugar de `a[31:2]`, lo que permite alinear la lectura a 2 bytes en lugar de a 4. Tercero, la salida siempre devuelve 32 bits concatenando dos halfwords consecutivos en orden little-endian: el halfword en la posición `idx` contiene los bits 15:0 y el halfword en `idx+1` contiene los bits 31:16.

Este diseño permite leer correctamente tanto instrucciones de 32 bits (que ocupan dos halfwords consecutivos) como instrucciones RVC de 16 bits (que ocupan un solo halfword, con el halfword siguiente correspondiente a la siguiente instrucción). La lógica del decompressor se encarga de interpretar correctamente cuál de los dos halfwords debe usar cuando encuentra una instrucción comprimida.

3.4.2 if_stage.v

Para este device del pipeline, la implementación base requería de ajustes. En el pipeline base, la etapa IF simplemente leía una instrucción completa de 32 bits desde la memoria y la pasaba directamente al registro IF/ID. El PC se

incrementaba siempre en 4 unidades. Con la extensión RVC es necesario interponer el decompressor entre la memoria y la salida de la etapa, y el incremento del PC debe ser de 2 si la instrucción es comprimida o de 4 si es estándar.

Código antes del cambio (pipeline base):

La etapa IF original solo contenía el registro del PC (flopr), la instancia de imem y el multiplexor de selección del próximo PC. No había decompressor ni lógica de detección de compresión, y el incremento era fijo en 4.

```
// Versión base -- sin decompressor, PC siempre +4
flopr #(32) pcreg (.clk(clk), .reset(reset), .d(PCNextF), .q(PCF));
imem #(.MEM_FILE(INSTR_MEM_FILE)) im (.a(PCF), .rd(InstrF));
assign PCPlus4F = PCF + 32'd4;
```

Código después del cambio (con soporte RVC):

```
wire [31:0] RawInstrF;
wire      IsCompressedF;
wire [31:0] DecomplInstrF;

flopr #(32) pcreg (.clk(clk), .reset(reset), .d(PCNextF), .q(PCF));
imem #(.MEM_FILE(INSTR_MEM_FILE)) im (.a(PCF), .rd(RawInstrF));

decompressor decomp (
    .instr16(RawInstrF[15:0]),
    .instr32(DecomplInstrF),
    .is_compressed(IsCompressedF)
);

assign InstrF    = IsCompressedF ? DecomplInstrF : RawInstrF;
assign PCIncF    = IsCompressedF ? (PCF + 32'd2) : (PCF + 32'd4);
assign PCPlus4F  = PCIncF;
```

Se agregaron tres señales internas: RawInstrF transporta la instrucción cruda leída de la memoria (que puede ser de 16 o 32 bits); IsCompressedF es la bandera que produce el decompressor cuando detecta que los bits instr[1:0] son distintos de 2'b11; y DecomplInstrF es la instrucción ya expandida a 32 bits.

La salida InstrF ahora se selecciona entre la instrucción expandida y la original mediante un multiplexor controlado por IsCompressedF. De esta manera, el resto del pipeline recibe siempre una instrucción de 32 bits compatible y no necesita conocer el tamaño original de la instrucción.

El incremento del PC, antes fijo en 4, ahora se calcula condicionalmente: si la instrucción es comprimida ($IsCompressedF = 1$), el PC avanza 2 bytes; si es estándar, avanza 4 bytes. Este valor se exporta como PCPlus4F al orquestador del pipeline, que lo propagará a las etapas siguientes para su uso en el cálculo de jal y jalr.

3.4.3 pipeline.v

De manera análoga, el módulo pipeline.v requirió de ajustes.

En el pipeline base, el valor PCPlus4 que viaja por los registros pipeline IF/ID, ID/EX, EX/MEM y MEM/WB se calculaba directamente como $PCF + 32'd4$ dentro del bloque siempre del registro IF/ID. Esta suma hardcodeda no funciona con RVC, donde el avance puede ser de 2 bytes.

Código antes del cambio (pipeline base):

```
// Declaración de señales en IF
wire [31:0] PCF, InstrF;

// En el registro IF/ID
PCPlus4D <= PCF + 32'd4;
```

El valor PCPlus4D se obtenía sumando 4 al PC actual dentro del propio bloque always de pipeline.v, sin pasar por la etapa IF. Cualquier variación en el incremento habría requerido modificar esta línea.

Código después del cambio (con soporte RVC):

```
// Declaración de señales en IF
wire [31:0] PCF, InstrF;

// En el registro IF/ID
PCPlus4D <= PCF + 32'd4;
wire [31:0] PCF, InstrF, PCPlus4F;

// Conexión del nuevo puerto en la instancia de if_stage
if_stage #(.INSTR_MEM_FILE(INSTR_MEM_FILE)) ifs (
    ...
    .PCPlus4F(PCPlus4F)
);

// En el registro IF/ID
PCPlus4D <= PCPlus4F;
```

Se realizaron tres cambios. Primero, se agregó el cable PCPlus4F a la declaración de señales de la etapa IF. Segundo, se conectó este cable al puerto del mismo nombre en la instancia de if_stage. Tercero, la asignación en el registro IF/ID dejó de usar la suma hardcodeada PCF + 32'd4 y pasó a tomar el valor directamente desde PCPlus4F, que ya contiene el incremento correcto (2 o 4) calculado dentro de if_stage.

Este cambio es sutil pero fundamental: permite que el valor PCPlus4 que se escribe en el registro destino tras un jal o jalr sea el valor correcto independientemente de si la instrucción que provocó el salto era de 16 o de 32 bits.

3.4.4 regfile.v

En el pipeline base, el banco de registros no se inicializaba explícitamente. En simulación, los registros comenzaban con el valor x (unknown), y como la lectura de x0 estaba hardcodeada a 0 mediante la asignación (a1 != 0) ? rf[a1] : 0, las instrucciones RV32I que usaban x0 como fuente funcionaban correctamente porque nunca leían un registro no inicializado. Sin embargo, las instrucciones RVC como c.addi rd, imm se expanden a addi rd, rd, imm, que lee el registro rd como fuente. Si rd no es x0 y además nunca ha sido escrito, su valor es x y el resultado de la operación propaga ese x a través del pipeline.

Código antes del cambio (pipeline base):

El banco de registros se declaraba sin ningún bloque de inicialización:

```
reg [31:0] rf[31:0];
```

Los registros comenzaban con contenido indeterminado en simulación.

Código después del cambio (con soporte RVC):

```
reg [31:0] rf[31:0];
integer i;
initial begin
    for (i = 0; i < 32; i = i + 1)
        rf[i] = 32'b0;
end
```

Se agregó un bloque inicial que, al comienzo de la simulación, recorre los 32 registros y les asigna el valor cero. Este bloque se ejecuta una sola vez antes de que comience la actividad del reloj, de modo que todos los registros parten de un estado conocido.

Es importante notar que este cambio es puramente funcional para la simulación y no modifica la semántica del hardware sintetizable. En una implementación física, los registros de la CPU arrancan con un valor indefinido, pero el software de inicialización (rutina de boot) debe escribir los registros antes de usarlos. La inicialización explícita en el modelo Verilog simplemente elimina la indeterminación en las pruebas, permitiendo verificar el comportamiento de las instrucciones RVC sin necesidad de un programa de inicialización previa.

3.4.5 Hazard unit

El módulo `hazard_unit.v` no requirió modificaciones para soportar la extensión RVC. Esto se debe a que el descompresor opera en la etapa IF; por lo tanto, cuando la instrucción cruza el registro de pipeline IF/ID y entra a la lógica de riesgos, ya ha sido completamente expandida a 32 bits. El bloque de control recibe señales y números de registros estándar, sin "saber" jamás si el formato original era de 16 o 32 bits.

Gracias a esta independencia, los mecanismos de *forwarding*, *stalling* por load-use y *flushing* por saltos funcionan de manera idéntica para ambas extensiones. La unidad de riesgos sigue operando sobre el espacio completo de 5 bits (`x0` a `x31`), ya que los registros restringidos (`x8` a `x15`) de las instrucciones RVC son traducidos a identificadores de 5 bits perfectamente válidos por el descompresor.

Esta transparencia arquitectónica es la mayor ventaja del diseño implementado. Al expandir la instrucción al inicio del *pipeline*, todo el resto del procesador permanece inalterado. Esto simplifica drásticamente el proceso de verificación del hardware y reduce el riesgo de introducir nuevos errores (*bugs*) en la compleja lógica de control del procesador segmentado.

4. Resultados

4.1 Programa de prueba

El programa de prueba `test_10_instrucciones.mem` fue diseñado para verificar el correcto funcionamiento de las 10 instrucciones RVC requeridas para esta entrega, combinándolas con instrucciones estándar RV32I de 32 bits en una misma secuencia de ejecución.

El programa consta de 17 instrucciones en total: 7 instrucciones RV32I (addi y beq) y 10 instrucciones RVC. En términos de memoria, esto representa 7×4 bytes + 10×2 bytes = **48 bytes** de código.

Estrategia de inicialización

En lugar de inicializar cada registro desde cero con `addi rd, x0, imm`, el programa utiliza una estrategia de encadenamiento: solo 4 registros se inicializan directamente desde x0, y los 3 restantes reutilizan resultados de instrucciones previas. Esto reduce el número de instrucciones de inicialización y además demuestra que el forwarding del procesador maneja correctamente dependencias entre instrucciones RV32I que leen valores producidos por instrucciones RVC anteriores. Las inicializaciones del código se muestran en la siguiente tabla:

Registro	Inicialización	Método
x8	<code>addi x8, x0, 10</code>	Desde x0
x9	<code>addi x9, x0, 3</code>	Desde x0
x11	<code>addi x11, x0, 7</code>	Desde x0
x14	<code>addi x14, x0, 8</code>	Desde x0
x10	<code>addi x10, x8, 7</code>	Reutiliza x8=13 (resultado de c.addi)
x12	<code>addi x12, x11, 5</code>	Reutiliza x11=7
x13	<code>addi x13, x11, 3</code>	Reutiliza x11=7

Por qué se eligieron estos valores

Los valores iniciales fueron seleccionados para que cada operación produzca un resultado único y verificable por simple inspección. Por ejemplo, 13 AND 7

es fácilmente verificable en binario ($1101 \& 0111 = 0101 = 5$), y la cadena de shifts sobre x14 ($8 > 32 > 4 > 2$) demuestra sucesivamente c.slli, c.srli y c.srai sin overflow ni pérdida de información.

Verificación automática

También se agregó en el testbench un comparador para los 8 registros de resultado contra sus valores esperados utilizando comparación exacta . Si todos coinciden, imprime PASS; de lo contrario, FAIL. Esto garantiza la verificación sin depender de interpretación visual del waveform.

La simulación ejecutada produjo el siguiente resultado:

```
None
=== Test E2 Condensado — 10 instrucciones RVC ===
--- Resultados ---
x8 = 13 (esperado: 13)
x9 = 16 (esperado: 16)
x10 = 5 (esperado: 5)
x11 = 7 (esperado: 7)
x12 = 4 (esperado: 4)
x13 = 10 (esperado: 10)
x14 = 2 (esperado: 2)
x15 = 8192 (esperado: 8192)
PASS
```

El programa termina con beq x0, x0, 0, un branch que salta a sí mismo generando un loop infinito. Esto garantiza que el procesador no avance hacia posiciones de memoria sin inicializar una vez completadas las instrucciones de prueba

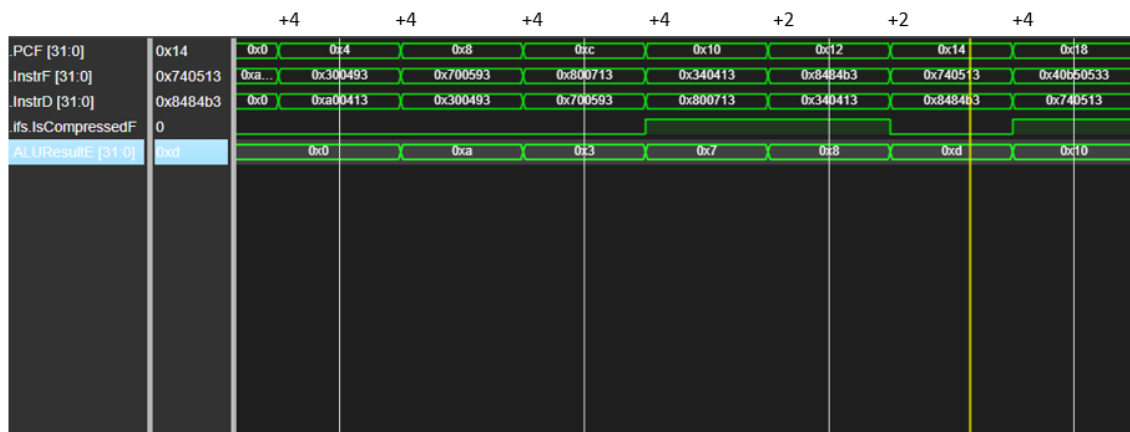
4.2 Análisis del waveform

El waveform captura cinco señales: PCF, InstrF, InstrD, IsCompressedF y ALUResultE, a continuación se mostrará el waveform completo obtenido y se explicará a detalle cada señal.



1. PCF — Program Counter

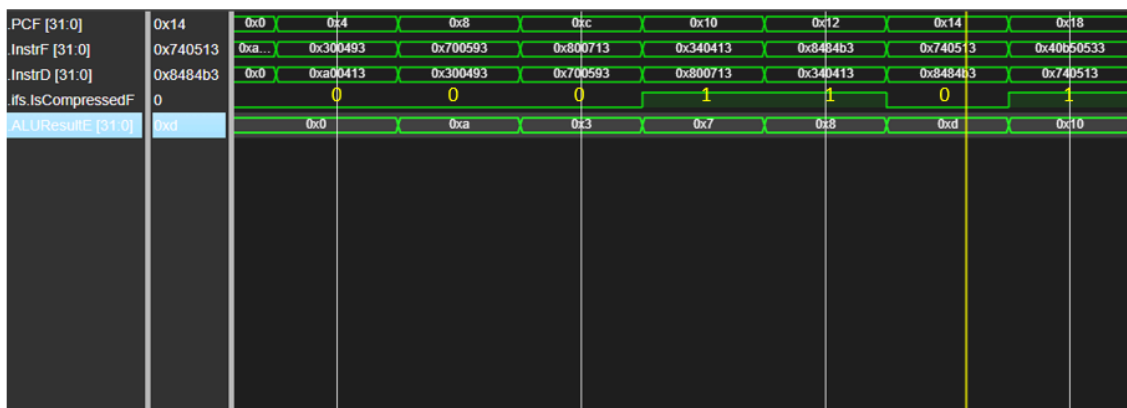
El PC demuestra los dos modos de avance del procesador extendido. Para instrucciones de 32 bits, el PC avanza de 4 en 4. A partir de 0x08, donde comienza la primera instrucción comprimida, el PC avanza de 2 en 2. Al encontrar nuevamente una instrucción de 32 bits en 0x14, vuelve al incremento de 4. Este patrón alternante +4 para RV32I, +2 para RVC es la evidencia directa de que el módulo decompressor detecta correctamente el tipo de instrucción y controla el adder PC+2 y PC+4 de la etapa IF.



2. IsCompressedF

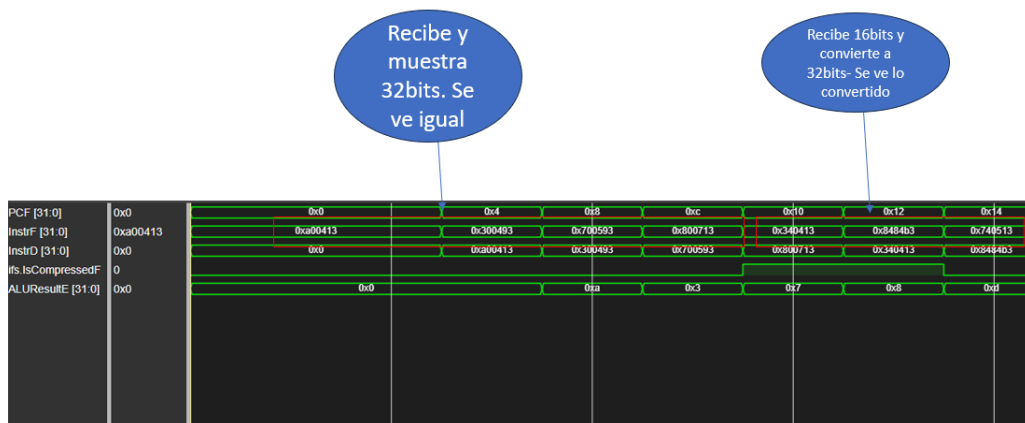
La señal IsCompressedF permanece en 0 durante las instrucciones de 32 bits y se eleva a 1 cada vez que el PC apunta a una instrucción RVC. En el waveform se observan dos grupos de pulsos altos: el primero en 0x10–0x12 que son las instrucciones c.addi y c.add, el segundo en 0x18–0x1A que son c.sub y c.and, y el último bloque continuo en 0x24 al 0x2E cubriendo c.or, c.xor, c.slli, c.srli, c.srai y c.lui.

Esta señal es generada combinacionalmente por el decompressor evaluando $\text{instr}[1:0] \neq 2'b11$.



3. InstrF — Instrucción expandida

InstrF muestra siempre la instrucción de 32 bits lista para el pipeline. Para instrucciones RV32I, InstrF es idéntica a los bits leídos de memoria. Para instrucciones RVC, InstrF contiene la versión expandida producida por el decompressor. Por ejemplo, cuando el PC vale 0x8, la memoria entrega 0x040D donde c.addi x8, 3 en 16 bits, pero InstrF muestra 0x00340413 que es addi x8, x8, 3 en 32 bits. El pipeline nunca ve la instrucción de 16 bits, solo trabaja con 32 bits.



4. InstrD — Instrucción en etapa ID

InstrD es InstrF con un ciclo de retraso, correspondiente al registro IF/ID. Se puede verificar que cada valor de InstrF aparece en InstrD exactamente un ciclo después, confirmando que el registro de pipeline propaga correctamente las instrucciones ya expandidas.



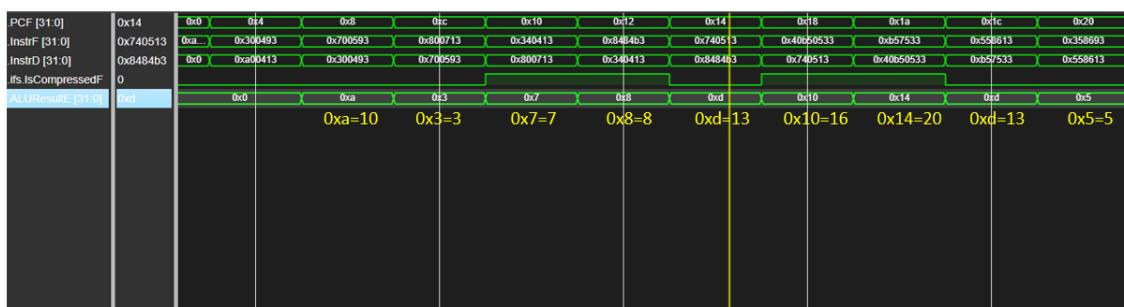
5. ALUResultE — Resultados de operaciones

La señal ALUResultE permite rastrear el resultado de cada instrucción en la etapa de ejecución. La tabla siguiente correlaciona cada valor observado con la instrucción que lo produce:

ALUResultE	Decimal	Instrucción	Operación
0xa	10	addi x8, x0, 10	inicialización
0x3	3	addi x9, x0, 3	inicialización
0x7	7	addi x11, x0, 7	inicialización

0x8	8	addi x14, x0, 8	inicialización
0xd	13	c.addi x8, 3	$10 + 3 = 13$
0x10	16	c.add x9, x8	$3 + 13 = 16$
0x14	20	addi x10, x8, 7	$13 + 7 = 20$
0xd	13	c.sub x10, x11	$20 - 7 = 13$
0x5	5	c.and x10, x11	$1101 \& 0111 = 0101$
0xc	12	addi x12, x11, 5	$7 + 5 = 12$
0xa	10	addi x13, x11, 3	$7 + 3 = 10$
0xe	14	c.or x12, x13	$1100 \mid 1010 = 1110$
0x4	4	c.xor x12, x13	$1110 \oplus 1010 = 0100$
0x20	32	c.slli x14, 2	$8 \ll 2 = 32$
0x4	4	c.srli x14, 3	$32 \gg 3 = 4$
0x2	2	c.srai x14, 1	$4 \gg 1 = 2$ (aritmético)
0x2000	8192	c.lui x15, 2	$2 \ll 12 = 8192$

Estos resultados los podemos comprobar en los primeros resultados de la misma waveform:



Para evidenciar que una instrucción comprimida atraviesa el pipeline igual que una de 32 bits, se sigue c.addi x8, 3 (PC = 0x08) por las cinco etapas: en IF la memoria entrega 0x040D y el decompressor la expande a 0x00340413 (addi x8, x8, 3); un ciclo después aparece en InstrD en ID, ya decodificada como una addi ordinaria; en EX la ALU calcula $10 + 3$ y ALUResultE da 0xD (13); pasa por MEM sin acceder a memoria; y en WB escribe 13 en x8. Esto confirma que, a partir de IF, el origen comprimido de la instrucción es invisible para el resto del pipeline.

5. Conclusiones

La implementación de la extensión RISC-V Compressed Instructions sobre el procesador pipelined de la Parte 1 demuestra que es posible incorporar soporte para instrucciones de 16 bits con cambios mínimos y quirúrgicos al datapath original, sin comprometer el correcto funcionamiento de ninguna de las instrucciones existentes de 32 bits.

Implementación

El núcleo de la solución es el módulo **decompressor**, ubicado en la etapa IF del pipeline, que actúa como una capa de traducción transparente. Su diseño como lógica combinacional pura sin estado ni clock garantiza que la expansión de 16 a 32 bits ocurre dentro del mismo ciclo de fetch, sin introducir latencia adicional al pipeline.

Desde la perspectiva de las etapas ID, EX, MEM y WB, todas las instrucciones llegan siempre como instrucciones de 32 bits; el hecho de que algunas hayan sido comprimidas en memoria es completamente invisible para el resto del procesador.

El segundo cambio fundamental fue en la memoria de instrucciones, que pasó de almacenar palabras de 32 bits a halfwords de 16 bits. Esto permitió al PC apuntar a cualquier dirección alineada a 2 bytes, habilitando que instrucciones de 16 y 32 bits convivan en el mismo espacio de memoria sin conflictos de alineación. El incremento del PC también fue adaptado para avanzar 2 bytes ante instrucciones comprimidas y 4 bytes ante instrucciones estándar, controlado por la señal `IsCompressedF` producida directamente por el decompressor.

Notablemente, la unidad de hazards no requirió ninguna modificación. Dado que el decompressor traduce las instrucciones antes de que entren al pipeline, los mecanismos de forwarding, stall por load-use y flush por branches operan sobre registros de 5 bits y señales de control estándar, exactamente igual que antes.

Sobre los resultados

La simulación del programa de prueba produjo los siguientes valores finales en los registros, verificados por el testbench:

Registro	Resultado	Instrucción responsable
x8	13	c.addi x8, 3 (10 + 3)
x9	16	c.add x9, x8 (3 + 13)
x10	5	c.and x10, x11 (13 & 7)
x11	7	addi x11, x0, 7 (sin cambio)
x12	4	c.xor x12, x13 (14 $\oplus$ 10)
x13	10	addi x13, x11, 3 (sin cambio)
x14	2	c.srai x14, 1 (4 >> 1 aritmético)

x15	8192	c.lui x15, 2 (2 << 12)
-----	------	------------------------

Todos los valores coinciden exactamente con los esperados. El testbench reportó PASS, confirmando el correcto funcionamiento de las 10 instrucciones RVC implementadas: c.addi, c.add, c.sub, c.and, c.or, c.xor, c.slli, c.srli, c.srai y c.lui.

El waveform permite corroborar visualmente cada etapa del proceso. La señal

IsCompressedF alterna entre 0 y 1 según el tipo de instrucción en fetch, lo cual se refleja directamente en el patrón del PC: los saltos de 4 bytes corresponden a instrucciones RV32I y los saltos de 2 bytes corresponden a instrucciones RVC.

La señal InstrF muestra siempre la instrucción expandida a 32 bits, y ALUResultE traza con precisión el resultado de cada operación, permitiendo rastrear paso a paso la cadena completa de cómputo.

Comparativa de tamaño y performance

El programa equivalente implementado únicamente con instrucciones RV32I de 32 bits requeriría $17 \text{ instrucciones} \times 4 \text{ bytes} = 68 \text{ bytes}$ de memoria de programa. La versión mixta RV32I + RVC, con 7 instrucciones de 32 bits y 10 instrucciones de 16 bits, ocupa $7 \times 4 + 10 \times 2 = 48 \text{ bytes}$ por lo que se obtuvo una reducción del 24% en el tamaño del programa sin modificar la lógica ni el número de operaciones realizadas.

Esta reducción tiene implicaciones prácticas importantes: en sistemas embebidos con memoria limitada, una densidad de código mayor permite almacenar programas más grandes en el mismo espacio, o reducir el costo del hardware de memoria.

Adicionalmente, un programa más compacto en memoria genera menos tráfico en el bus de instrucciones, lo que puede traducirse en mejor aprovechamiento del ancho de banda en sistemas con jerarquía de caché.

En cuanto al performance en ciclos de reloj, ambas versiones ejecutan el mismo número de instrucciones y ciclos, ya que el decompressor no agrega ciclos adicionales. La ganancia de esta extensión es exclusivamente en densidad de código, no en velocidad de ejecución por instrucción.

En conclusión, la extensión RVC se integra de forma efectiva al pipeline existente: un solo módulo combinacional en la etapa IF es suficiente para habilitar soporte completo para instrucciones de 16 bits, preservando toda la lógica de hazards, control y ejecución sin modificación alguna, y logrando una reducción significativa en el tamaño del código generado.